

Performance Evaluation of ESPCN-Based Image Super-Resolution on Raspberry Pi 5 and Hailo-8L

Federico Longobardi
Department of Industrial Engineering
University of Trento
Trento, Italy
federico.longobardi@studenti.unitn.it

Abstract—This work presents the development and evaluation of an image super-resolution pipeline targeting real-time execution on edge hardware. The proposed system is designed to perform $\times 2$ image upscaling while targeting a throughput of at least 15 frames per second (FPS), making it suitable for embedded and robotic applications.

Traditional interpolation methods, including Nearest Neighbor and Bicubic interpolation, are compared against the Efficient Sub-Pixel Convolutional Neural Network (ESPCN), a convolutional neural network specifically designed for efficient image super-resolution. Experiments are conducted on a Raspberry Pi 5 platform using grayscale images at multiple input resolutions. Reconstruction quality is evaluated through Peak Signal-to-Noise Ratio (PSNR), while computational efficiency is assessed in terms of latency, FPS, current consumption (mA), power consumption (W), and energy per frame (J/frame).

The obtained results provide insights into the trade-offs between image quality, computational performance, and energy efficiency, contributing to the assessment of deep learning-based super-resolution for low-power embedded systems and edge AI applications.

Index Terms—Image Super-Resolution, TinyML, Edge AI, Raspberry Pi 5, Hailo-8L, ESPCN, Benchmarking

I. INTRODUCTION

Image super-resolution (SR) is a computer vision technique aimed at reconstructing high-resolution images from low-resolution inputs. The field has attracted significant attention in recent years due to its applications in robotics, surveillance, and edge AI systems, where high-quality visual information is often required despite limited sensing and computational resources [1].

Traditional image upscaling methods, such as Nearest Neighbor and Bicubic interpolation, are computationally efficient and widely used in embedded systems. However, these approaches often fail to recover high-frequency image details, resulting in blurry reconstructions. Deep learning methods based on convolutional neural networks (CNNs) have significantly improved reconstruction quality and have become the state of the art in image super-resolution tasks [2].

Despite their effectiveness, deploying deep learning models on edge devices remains challenging due to limited computational resources, memory constraints, and power consumption requirements. Dedicated AI accelerators have recently emerged as a promising solution to enable real-time inference on embedded platforms. Consequently, lightweight architec-

tures specifically designed for efficient inference have become increasingly important for embedded applications.

This work investigates the feasibility of deploying image super-resolution on edge hardware by comparing traditional interpolation techniques with the Efficient Sub-Pixel Convolutional Neural Network (ESPCN) [3], a lightweight CNN architecture based on sub-pixel convolution. The proposed pipeline targets a $\times 2$ upscaling factor while aiming to satisfy a real-time requirement of at least 15 FPS on embedded hardware.

Experiments are conducted on a Raspberry Pi 5 equipped with a Hailo-8L accelerator and a Camera Module 3. The evaluated methods are compared in terms of reconstruction quality, computational performance, and energy efficiency using metrics such as PSNR, latency, FPS, current consumption, power consumption, and energy per frame.

The obtained results contribute to the evaluation of the feasibility of real-time deep learning-based super-resolution on low-power embedded systems and provide insights into the trade-offs between image quality, computational performance, and energy efficiency.

II. EXPERIMENTAL SETUP

A. Hardware Platform

The experimental platform consists of a Raspberry Pi 5 equipped with 8 GB of RAM, representing a modern low-power edge computing device. Neural network acceleration is provided by the Hailo-8L AI accelerator, enabling efficient inference on dedicated hardware. Image acquisition for the final demonstration is performed using the Raspberry Pi Camera Module 3.

Electrical measurements are obtained through an external USB power meter, allowing the evaluation of current consumption and power usage during inference. The target application involves real-time image super-resolution for systems.

B. Software Environment

The benchmarking framework was implemented in Python. PyTorch was used for neural network inference and model evaluation, while OpenCV was employed for image processing operations such as resizing and image manipulation. The ESPCN model was exported to ONNX format and used for both benchmarking and deployment on the target edge platforms.

C. Dataset

The DIV2K dataset was divided into two subsets. The first subset was used for fine-tuning the ESPCN model, while a second subset of 30 images was reserved exclusively for quantitative evaluation. This separation was adopted to avoid evaluating the model on the same images used during fine-tuning.

To match the input requirements of the implemented ESPCN model, all images were converted to grayscale before processing. Since the evaluated super-resolution factor is $\times 2$, high-resolution reference patches were first extracted from the original images. Each high-resolution patch was then downsampled by a factor of two to generate the corresponding low-resolution input.

For example, for an input resolution of 640×640 , a high-resolution crop of 1280×1280 was extracted from the original image and then downsampled to 640×640 . The super-resolution output was finally compared against the original 1280×1280 crop.

The evaluated image resolutions were:

- $64 \times 64 \rightarrow 128 \times 128$
- $112 \times 112 \rightarrow 224 \times 224$
- $224 \times 224 \rightarrow 448 \times 448$
- $480 \times 640 \rightarrow 960 \times 1280$
- $640 \times 640 \rightarrow 1280 \times 1280$

D. Image Processing Pipeline

The proposed pipeline performs image super-resolution with an upscaling factor of $\times 2$. For each experiment, a high-resolution crop matching the target output size was first extracted from the original DIV2K image. This crop was used as the ground-truth reference image. A corresponding low-resolution input was then generated by downsampling the crop by a factor of two.

The low-resolution image was processed by the selected super-resolution algorithm to reconstruct a high-resolution output image. The reconstructed output was then compared with the original high-resolution crop to evaluate reconstruction quality. Computational performance and energy consumption were measured during the inference stage.

Figure 1 summarizes the complete experimental workflow, including dataset preparation, low-resolution image generation, super-resolution inference, and quantitative evaluation.

E. Evaluation Metrics

The proposed methods were evaluated in terms of reconstruction quality, computational performance, and energy efficiency in order to provide a comprehensive assessment of their suitability for real-time edge deployment.

Image reconstruction quality was evaluated using the Mean Squared Error (MSE) and the Peak Signal-to-Noise Ratio (PSNR), two widely adopted metrics in image super-resolution literature [1]. The MSE measures the average squared difference between the reconstructed image and the corresponding ground-truth image, and is defined as

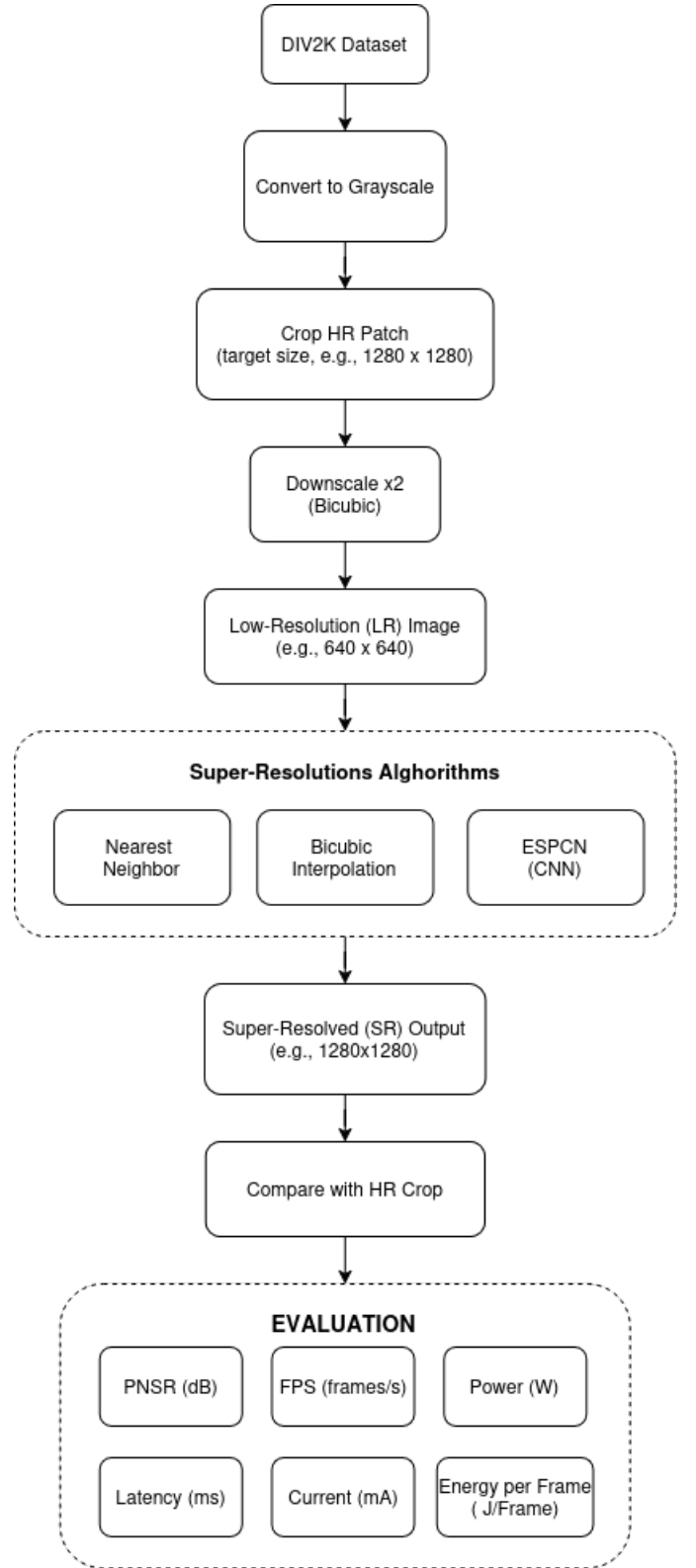


Fig. 1. Experimental workflow adopted in this work. A high-resolution crop is extracted from each DIV2K image, downsampled to generate the low-resolution input, processed by the evaluated super-resolution algorithm, and finally compared with the original crop using quality, performance, and energy metrics.

$$MSE = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (I(i, j) - K(i, j))^2 \quad (1)$$

where I denotes the ground-truth image, K the reconstructed image, and M and N the image dimensions.

The PSNR is computed as

$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \quad (2)$$

where MAX_I represents the maximum possible pixel value. Higher PSNR values generally indicate better reconstruction quality.

Computational performance was evaluated through inference latency and frames per second (FPS). Latency represents the average time required to process a single image, while FPS measures the maximum throughput achievable by the complete super-resolution pipeline. Since the target application requires real-time execution, a target throughput of at least 15 FPS was considered.

Energy efficiency was assessed by measuring current consumption (mA), power consumption (W), and energy per frame (J/frame). The latter was computed as

$$E_{frame} = \frac{P}{FPS} \quad (3)$$

where P denotes the average power consumption measured during inference. Energy per frame provides a hardware-independent indicator of the energy required to process a single image and is particularly relevant for comparing embedded implementations.

To improve the statistical reliability of the measurements, each experiment was repeated 30 times and the reported values correspond to the arithmetic mean of the collected measurements.

III. SUPER-RESOLUTION METHODS

This section describes the image upscaling methods evaluated in this work together with the preparation of the selected deep learning model for deployment on the target embedded platforms.

A. Baseline Methods

Nearest Neighbor and Bicubic interpolation were selected as baseline methods to provide a reference for evaluating the performance of the proposed deep learning approach.

Nearest Neighbor performs image upscaling by assigning each output pixel the value of its closest input pixel, resulting in minimal computational cost but limited reconstruction quality. Bicubic interpolation estimates each output pixel using a weighted combination of neighboring pixels, generally producing smoother images and improved visual quality. Due to its favorable trade-off between computational complexity and reconstruction accuracy, Bicubic interpolation is widely adopted as a reference method in image super-resolution research [1].

B. ESPCN Architecture

The Efficient Sub-Pixel Convolutional Neural Network (ESPCN), proposed by Shi et al. [3], was selected as the deep learning model evaluated in this work. Unlike conventional CNN-based super-resolution methods, ESPCN performs feature extraction directly in the low-resolution domain and reconstructs the final high-resolution image using a Pixel Shuffle operation. By postponing the upscaling step until the end of the network, most computations are carried out on low-resolution feature maps, significantly reducing computational complexity while maintaining competitive reconstruction quality. This characteristic makes ESPCN particularly suitable for real-time deployment on resource-constrained embedded platforms such as the Raspberry Pi 5 and the Hailo-8L accelerator.

C. Model Preparation

A pre-trained ESPCN model based on the original work of Shi et al. was adopted using the PyTorch implementation released by Lornatang [9]. The model was subsequently fine-tuned on a subset of the DIV2K dataset to better adapt the pre-trained weights to the target application.

Since the adopted implementation operates on single-channel images, all RGB images were converted to grayscale before both fine-tuning and evaluation.

After fine-tuning, the trained PyTorch model was exported to the ONNX format. The exported model was then used for both performance benchmarking and deployment on the target embedded platforms.

D. Deployment Pipeline

The exported ONNX model was first benchmarked on the Raspberry Pi 5 to evaluate inference latency, throughput, and energy consumption on a general-purpose embedded processor. Subsequently, the model was compiled for execution on the Hailo-8L AI accelerator to assess the benefits of dedicated neural network acceleration.

Finally, the optimized model was integrated into a real-time image super-resolution pipeline using the Raspberry Pi Camera Module 3, enabling live image acquisition and processing on the target hardware.

IV. RESULTS AND DISCUSSION

This section presents the experimental evaluation of the proposed image super-resolution pipeline. The analysis focuses on three key aspects: reconstruction quality, computational performance, and energy efficiency. The benchmark compares traditional interpolation methods with the proposed ESPCN model executed using different inference backends, including PyTorch, ONNX Runtime, and the Hailo-8L accelerator. Unless otherwise specified, the reported values correspond to the average obtained over the valid images of the DIV2K validation dataset.

TABLE I
AVERAGE RECONSTRUCTION QUALITY (PSNR) OBTAINED FOR THE
EVALUATED SUPER-RESOLUTION METHODS.

Method	Resolution	PSNR (dB)	Std (dB)
Nearest	64 × 64	28.07	5.57
Bicubic	64 × 64	30.30	6.03
ESPCN (PyTorch)	64 × 64	30.94	5.98
ESPCN (ONNX)	64 × 64	30.80	5.78
Nearest	112 × 112	27.67	4.62
Bicubic	112 × 112	30.00	5.26
ESPCN (PyTorch)	112 × 112	30.70	5.42
ESPCN (ONNX)	112 × 112	30.60	5.33
Nearest	224 × 224	27.44	3.98
Bicubic	224 × 224	29.81	4.80
ESPCN (PyTorch)	224 × 224	30.56	5.08
ESPCN (ONNX)	224 × 224	30.45	4.98
ESPCN (Hailo)	224 × 224	30.29	4.78
Nearest	480 × 640	28.14	3.99
Bicubic	480 × 640	30.63	4.78
ESPCN (PyTorch)	480 × 640	31.34	5.03
ESPCN (ONNX)	480 × 640	31.21	4.90
Nearest	640 × 640	28.49	3.91
Bicubic	640 × 640	31.04	4.69
ESPCN (PyTorch)	640 × 640	31.73	4.90
ESPCN (ONNX)	640 × 640	31.60	4.75
ESPCN (Hailo)	640 × 640	29.36	3.19

A. Reconstruction Quality

Table I summarizes the reconstruction quality obtained by the evaluated methods in terms of Peak Signal-to-Noise Ratio (PSNR).

As expected, Nearest Neighbor interpolation consistently provides the lowest reconstruction quality at every evaluated resolution. Although computationally inexpensive, the method introduces visible block artifacts and is unable to recover fine image details. Bicubic interpolation significantly improves the reconstruction quality over Nearest Neighbor by exploiting information from neighboring pixels. Across all evaluated resolutions, Bicubic increases the average PSNR by approximately 2-3 dB, confirming its effectiveness as a classical interpolation baseline. The ESPCN model further improves the reconstruction quality obtained with Bicubic interpolation. The PyTorch implementation achieves the highest PSNR values at all evaluated resolutions, with improvements ranging from approximately 0.6 to 0.8 dB over Bicubic interpolation. Although numerically modest, these improvements correspond to a more accurate reconstruction of image structures and edges. The ONNX implementation produces reconstruction quality very close to the original PyTorch model. The maximum PSNR difference is approximately 0.15 dB, indicating that exporting the network to the ONNX format introduces virtually no degradation in image quality while enabling a more efficient deployment on embedded hardware. For the Hailo implementation, the optimized 224×224 model achieves a PSNR comparable to both the PyTorch and ONNX implementations, confirming that hardware acceleration preserves most of the reconstruction capability. The lower PSNR observed for the 640×640 Hailo model is explained by the compilation process. While the 224×224 model was successfully compiled using Hailo optimization level 2, the 640×640 network could only

be compiled using optimization level 0 after higher optimization levels failed. Consequently, the 640×640 Hailo results should be interpreted as a functional deployment rather than a fully optimized implementation.

B. Computational Performance

The computational performance of the evaluated methods is summarized in Table II.

TABLE II
INFERENCE PERFORMANCE OF THE EVALUATED IMPLEMENTATIONS.

Method	Resolution	FPS	Std
Nearest	64 × 64	59201.96	555.34
Bicubic	64 × 64	16639.50	55.54
ESPCN (PyTorch)	64 × 64	187.62	0.69
ESPCN (ONNX)	64 × 64	445.88	3.31
Nearest	112 × 112	22086.85	174.70
Bicubic	112 × 112	6361.94	17.95
ESPCN (PyTorch)	112 × 112	57.73	0.26
ESPCN (ONNX)	112 × 112	120.06	0.29
Nearest	224 × 224	16426.40	114.85
Bicubic	224 × 224	4555.33	10.72
ESPCN (PyTorch)	224 × 224	12.63	0.01
ESPCN (ONNX)	224 × 224	28.98	0.03
ESPCN (Hailo)	224 × 224	566.73	1.76
Nearest	480 × 640	3614.86	6.42
Bicubic	480 × 640	1030.18	1.88
ESPCN (PyTorch)	480 × 640	2.13	0.02
ESPCN (ONNX)	480 × 640	4.65	0.01
Nearest	640 × 640	2580.04	12.17
Bicubic	640 × 640	746.04	1.36
ESPCN (PyTorch)	640 × 640	1.54	0.01
ESPCN (ONNX)	640 × 640	3.14	0.01
ESPCN (Hailo)	640 × 640	79.59	0.78

Traditional interpolation methods achieve extremely high throughput because they only perform image resizing operations. Even at the largest evaluated resolution, both Nearest Neighbor and Bicubic process hundreds of frames per second on the Raspberry Pi 5. The computational cost of ESPCN is considerably higher due to the convolutional operations required by the neural network. Nevertheless, the ONNX Runtime implementation consistently outperforms the original PyTorch execution across all evaluated resolutions. For example, at an input resolution of 224×224, the throughput increases from 12.63 FPS to 28.98 FPS, allowing the Raspberry Pi 5 to exceed the target real-time requirement of 15 FPS. The largest performance improvement is obtained by deploying the network on the Hailo-8L accelerator. At 224×224, the Hailo implementation reaches more than 560 FPS while maintaining reconstruction quality comparable to the CPU implementations. Even for the 640×640 model, despite the lower optimization level, the accelerator achieves nearly 80 FPS, representing a substantial improvement over both PyTorch and ONNX Runtime. Overall, these results demonstrate that the choice of the inference backend has a much larger impact on execution speed than the neural network architecture itself. While the model remains unchanged, hardware-aware deployment significantly improves computational performance.

C. Energy Efficiency

Energy efficiency is an important requirement for embedded applications, where computational resources are often constrained by power availability. Table III reports the measured current consumption, power consumption, and energy per processed frame.

TABLE III
POWER CONSUMPTION MEASUREMENTS ON THE RASPBERRY PI 5.

Platform	Res.	I (A)	P (W)	E (J/frame)
Idle	—	0.48	2.46	—
Bicubic	64 × 64	0.78	3.99	0.000240
Bicubic	112 × 112	0.81	4.13	0.000649
Bicubic	224 × 224	0.89	4.57	0.001003
Bicubic	480 × 640	1.03	5.30	0.005145
Bicubic	640 × 640	1.11	5.67	0.007600
ONNX	64 × 64	1.59	8.15	0.018279
ONNX	112 × 112	1.80	9.27	0.077212
ONNX	224 × 224	1.98	10.19	0.351622
ONNX	480 × 640	2.06	10.58	2.275269
ONNX	640 × 640	2.01	10.31	3.283439
Hailo	224 × 224	0.87	4.45	0.007852
Hailo	640 × 640	0.93	4.74	0.059555

The idle measurement provides a reference for the baseline power consumption of the Raspberry Pi 5 before executing any image processing task. Bicubic interpolation exhibits the lowest energy consumption per processed frame among the evaluated methods. This result is expected because the interpolation algorithm performs only simple image resizing operations while maintaining extremely high throughput. The ONNX implementation of ESPCN requires higher computational resources, resulting in increased power consumption and a rapid increase in energy per frame as the input resolution grows. At the highest evaluated resolution, the reduced throughput leads to an energy cost exceeding 3 J per processed frame. The Hailo accelerator significantly improves the overall energy efficiency of the system. Although the accelerator consumes additional power, the substantial increase in throughput more than compensates for this overhead. At 224×224, the energy required to process a single frame is reduced by more than one order of magnitude compared to the ONNX implementation executed on the Raspberry Pi CPU. A similar trend is observed at 640×640, confirming the benefits of dedicated neural network acceleration for edge AI applications.

D. Discussion

The experimental results highlight the trade-off between reconstruction quality, computational performance, and energy efficiency. Traditional interpolation methods provide extremely high throughput and low energy consumption but are limited in reconstruction quality. In contrast, ESPCN consistently produces higher PSNR values, demonstrating the effectiveness of lightweight deep learning models for single-image super-resolution. From the deployment perspective, exporting the model from PyTorch to ONNX preserves reconstruction quality while providing a substantial improvement in inference speed. This confirms that ONNX Runtime represents an effective optimization strategy for CPU-based embedded

inference. The largest benefits are obtained through dedicated hardware acceleration. The Hailo-8L implementation achieves real-time performance with a comfortable margin while maintaining reconstruction quality close to the original model. Furthermore, the accelerator considerably reduces the energy required to process each frame, making the proposed pipeline particularly suitable for low-power edge devices. One practical limitation observed during deployment concerns the compilation process of the Hailo models. Since HEF files are generated for fixed input resolutions, separate compiled models are required for different operating resolutions. Moreover, the achievable optimization level depends on the network configuration, as demonstrated by the 640×640 implementation, which required compilation using optimization level 0. Despite these limitations, the obtained results demonstrate the feasibility of deploying lightweight image super-resolution models on embedded platforms for real-time applications.

E. Real-Time Demonstration

To validate the practical deployment of the proposed super-resolution system, a live demonstration was implemented using a Raspberry Pi 5 equipped with a Hailo-8L AI accelerator and a Raspberry Pi Camera Module 3. Unlike the offline benchmark presented in the previous section, which evaluates only the inference performance on a static dataset, the live demonstration measures the complete end-to-end processing pipeline under realistic operating conditions.

Each frame acquired by the camera is converted to grayscale and directly provided to the ESPCN model running on the Hailo-8L accelerator. The reconstructed ×2 super-resolved image is then encoded as an MJPEG stream and transmitted to a remote computer for real-time visualization. The processing pipeline is summarized as follows:

Camera Module 3 → Grayscale Conversion → ESPCN
(Hailo-8L) → Super-Resolved Image → MJPEG Streaming
→ Remote Display

During execution, the system continuously profiles each stage of the pipeline, including image acquisition, preprocessing, Hailo inference, post-processing, JPEG encoding, CPU utilization, memory consumption, and the resulting end-to-end frame rate. This allows the computational bottlenecks of the complete embedded vision pipeline to be identified.

Fig. 2 illustrates the hardware configuration adopted for the live demonstration, while Fig. 3 shows a screenshot of the running application.

As shown in Table IV, the end-to-end frame rate is lower than the standalone Hailo benchmark reported in the previous section. This behavior is expected because the live demonstration includes additional operations such as image acquisition, preprocessing, JPEG encoding, network transmission, and visualization. Profiling results indicate that image acquisition represents the dominant source of latency, whereas the Hailo inference requires only a few milliseconds per frame. These results demonstrate that, in practical embedded vision systems,



Fig. 2. Experimental setup used for the real-time demonstration. The Raspberry Pi Camera Module 3 acquires live images, which are processed by the Raspberry Pi 5 equipped with the Hailo-8L accelerator and streamed to a remote computer for visualization.

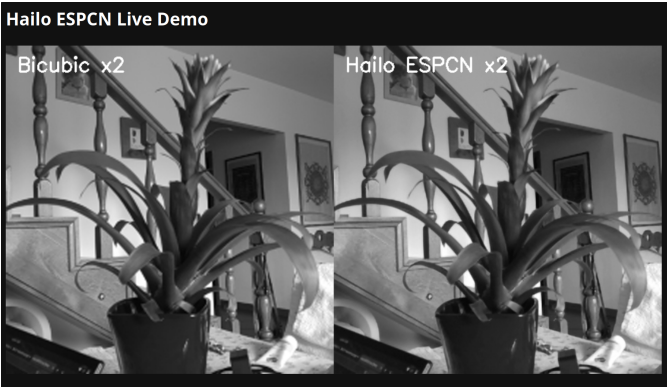


Fig. 3. Comparison between bicubic interpolation (left) and the corresponding $\times 2$ super-resolved output generated by the ESPCN model running on the Hailo-8L accelerator (right).

TABLE IV
AVERAGE EXECUTION TIME OF THE LIVE DEMONSTRATION PIPELINE (224×224 MODEL).

Stage	Time (ms)
Image acquisition	24.91
Pre-processing	0.19
Hailo inference	2.15
Post-processing	2.12
JPEG encoding	3.89
Total	33.27
FPS	30.06

the overall throughput depends on the complete processing pipeline rather than on the neural network inference alone.

The measured CPU utilization remained below one third of the available processing capacity throughout the live demonstration at 640×640 resolution, while the memory footprint was approximately 170 MB. This indicates that the Hailo-8L accelerator effectively offloads the computationally intensive super-resolution task from the Raspberry Pi CPU, leaving sufficient computational resources available for additional application-level functionalities. This demonstrates one of the main advantages of dedicated AI accelerators in embedded systems, namely the ability to maintain real-time performance while preserving computational resources for other concurrent tasks.

V. CONCLUSION

The presented work developed and evaluated a lightweight image super-resolution pipeline for edge AI applications based on the ESPCN architecture. Traditional interpolation methods, namely Nearest Neighbor and Bicubic interpolation, were systematically compared with a deep learning approach across multiple input resolutions using reconstruction quality, inference performance, and energy efficiency as evaluation metrics.

Experimental results demonstrated that ESPCN consistently provides higher reconstruction quality than conventional interpolation while maintaining computational requirements compatible with embedded deployment. The use of ONNX Runtime significantly reduced inference latency with respect to the native PyTorch implementation, whereas the Hailo-8L accelerator enabled real-time execution with substantially higher throughput and lower energy consumption. Furthermore, the live demonstration confirmed the feasibility of deploying the complete super-resolution pipeline on a Raspberry Pi 5, showing that the overall system performance is primarily limited by image acquisition and data transfer rather than by neural network inference itself.

Overall, the obtained results highlight the effectiveness of combining lightweight super-resolution models with dedicated AI accelerators for real-time embedded vision applications. The proposed framework provides a practical reference for evaluating the trade-off between reconstruction quality, computational performance, and energy efficiency on resource-constrained platforms.

Future work may include fine-tuning the network for specific camera sensors, extending the pipeline to full-color super-

resolution, evaluating additional lightweight super-resolution architectures, and investigating further optimizations of the image acquisition and streaming stages to improve the end-to-end system throughput.

REFERENCES

- [1] L. Wang et al., “Deep Learning for Image Super-Resolution: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3365–3387, 2021.
- [2] C. Dong, C. C. Loy, K. He, and X. Tang, “Image Super-Resolution Using Deep Convolutional Networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.
- [3] W. Shi et al., “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1874–1883.
- [4] R. Timofte et al., “NTIRE 2017 Challenge on Single Image Super-Resolution: Methods and Results,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017.
- [5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [6] Raspberry Pi Foundation, “Raspberry Pi 5 Documentation,” 2025.
- [7] Hailo Technologies, “Hailo-8L AI Accelerator Documentation,” 2025.
- [8] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [9] Lornatang, “ESPCN-PyTorch,” GitHub repository, 2024. [Online]. Available: <https://github.com/Lornatang/ESPCN-PyTorch>